WEEK 3 · LESSON 18 OF 24 · HANDOUT

Week 3 Project: A Responsible AI Audit

Study time: **about 120 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=FkCb7345Hj8 — open the lesson online to watch inside the course.

🎓 **Go deeper:** SciPy 2023: Responsible AI in Practice (Rumman Chowdhury) — https://www.youtube.com/watch?v=dQHjntrqrFQ

🎓 **Go deeper:** PyCon 2019: Measuring Model Fairness — https://www.youtube.com/watch?v=pYXz507ths0

Your goal in this lesson: Audit a deployed model for performance, fairness, explainability, privacy and security, and produce a written audit report with findings and recommendations.

The brief

The SACCO board asks: "Is our loan model safe and fair to use?" You will run a complete audit and write the report the board will read. The audit must be **repeatable**: one function that can run again after every retraining.

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

```
def group_report(data, group, decision, outcome="repaid"):
    rows = []
    for name, g in data.groupby(group):
        rows.append({
            group: name, "people": len(g),
            "approval_rate": g[decision].mean(),                              # selection rate
            "approved_if_would_repay": g.loc[g[outcome] == 1, decision].mean(), # true positive
rate
            "approved_if_would_default": g.loc[g[outcome] == 0, decision].mean(), # false
positive rate
            "repay_rate": g[outcome].mean(),
        })
    return pd.DataFrame(rows).set_index(group).round(3)

def fairness_summary(report):
    a, t = report["approval_rate"], report["approved_if_would_repay"]
    return {"approval_gap": round(float(a.max() - a.min()), 3),
            "disparate_impact": round(float(a.min() / a.max()), 3),
            "equal_opportunity_gap": round(float(t.max() - t.min()), 3)}
```

Part 1 — Performance

```
p = model.predict_proba(test[FEATURES])[:, 1]
audit = test.assign(score=p, approved=(p >= 0.5).astype(int))
performance = {"AUC": round(roc_auc_score(audit[TARGET], p), 3),
               "accuracy": round(accuracy_score(audit[TARGET], audit["approved"]), 3),
               "approval rate": round(float(audit["approved"].mean()), 3)}
print(performance)
```

Part 2 — Fairness

```
fair = {}
for group in ["area", "gender"]:
    report = group_report(audit, group, "approved")
    fair[group] = (report, fairness_summary(report))
    print(report.to_string()); print(fair[group][1], "\n")
```

Part 3 — Explainability

```
from sklearn.inspection import permutation_importance
perm = permutation_importance(model, test[FEATURES], test[TARGET], scoring="roc_auc",
n_repeats=5, random_state=0)
drivers = pd.Series(perm.importances_mean,
index=FEATURES).sort_values(ascending=False).round(3)
print(drivers.to_string())
```

Part 4 — Privacy and security checks

```
SENSITIVE = {"full_name", "phone", "national_id", "gender", "area", "age"}
checks = {
    "model uses no sensitive or identifying columns": not (set(FEATURES) & SENSITIVE),
    "fairness gap on equal opportunity below 0.10 for every group":
        all(s["equal_opportunity_gap"] < 0.10 for _, s in fair.values()),
    "disparate impact at least 0.8 for every group": all(s["disparate_impact"] >= 0.8 for _, s
in fair.values()),
    "top drivers are financial, not proxies": drivers.index[0] in {"income_rwf", "loan_rwf",
"late_payments"},
    "API requires a key and rate limit (Lesson 17)": True,          # confirm from the service
code
    "prediction logs pseudonymised (Lesson 9)": True,               # confirm from the logging
code
}
for name, ok in checks.items():
    print(("PASS  " if ok else "ISSUE ") + name)
```

Part 5 — Write the report

```
def table(df):
    return "\n".join("    " + line for line in df.to_string().splitlines())   # indented =
shown as a table

issues = [name for name, ok in checks.items() if not ok]
lines = ["# Loan model audit report", "",
         f"Model: logistic regression, features: {', '.join(FEATURES)}", "",
         "## 1. Performance", *[f"- {k}: {v}" for k, v in performance.items()], "",
         "## 2. Fairness"]
for group, (report, summary) in fair.items():
    lines += [f"### By {group}", table(report), "", f"Summary: {summary}", ""]
lines += ["## 3. What drives the model", table(drivers.to_frame("AUC lost when shuffled")), "",
          "## 4. Checks", *[f"- [{'x' if ok else ' '}] {n}" for n, ok in checks.items()], "",
          "## 5. Findings and recommendations"]
if issues:
    lines += [f"- ISSUE: {i}" for i in issues]
    lines += ["- Recommendation: keep human review for refused rural applicants until the gap
is reduced.",
              "- Recommendation: test mitigations from Lesson 14 and re-run this audit.",
              "- Recommendation: re-audit monthly in production and after every retraining."]
else:
    lines += ["- No blocking issues found. Re-audit monthly and after every retraining."]
open("audit_report.md", "w").write("\n".join(lines))
print("\n".join(lines[:12])); print("...")
print("issues:", issues)
```

The audit finds that the model is accurate and does not use sensitive data, but it **fails** the disparate-impact check for area: rural applicants are approved at only about 76% of the urban rate, below the 0.8 warning line. Rural applicants who would repay are also refused a little more often (a gap of about 5 points, inside our 0.10 limit). That is exactly the kind of finding a board must see before deciding.

Your report (three pages)

1. Summary for the board in five sentences: is the model safe to use, under which conditions?
2. Performance, fairness (by area and gender), drivers — with tables.
3. Privacy and security status.
4. Findings ranked by severity, each with an owner and a date.
5. Recommendation: deploy, deploy with conditions (for example human review), or do not deploy.

Marking guide (self-check)

Area	Excellent work
Coverage	Performance, fairness, explanations, privacy and security all checked
Evidence	Every claim backed by a number or a test
Honesty	Problems reported clearly, not hidden
Recommendations	Concrete, owned, dated
Repeatability	The audit runs again with one call

Remember this: A responsible AI audit is repeatable, evidence-based and honest — and its findings lead to concrete decisions, not just a report on a shelf.

Name: _____ Date: _____

Activity

Task 1

Complete Parts 1–5 and the board report. Then re-run the audit for the "train on real repayment, no mobile-money feature" model from Lesson 14 and compare the two reports side by side.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why must an audit be repeatable?
   - A. To make it longer
   - B. It is not needed
   - C. So it can run again after every retraining and in production
   - D. Boards like code
2. What did the audit find for area?
   - A. The model uses area as an input
   - B. No rural applicants
   - C. Perfect fairness
   - D. Rural applicants are approved at under 80% of the urban rate
3. Which is a good recommendation format?
   - A. 'Be fairer'
   - B. No recommendations
   - C. Only technical words
   - D. A concrete action with an owner and a date
4. What should happen to problems found in an audit?
   - A. Hide them
   - B. Report them clearly with evidence
   - C. Retrain until they disappear silently
   - D. Delete the data
5. Which option can be safer than full deployment or no deployment?
   - A. Deploy with conditions, such as human review of refused rural applicants
   - B. Remove the audit
   - C. Turn off logging
   - D. Deploy to everyone instantly
6. What should every audit claim be backed by?
   - A. Opinions
   - B. A number or a test
   - C. Rumours
   - D. Colours

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.